



מכון טכנולוגי חולון
Holon Institute of Technology

מבני נתונים

תרגיל 3 – מחסנית ותור

פתרונות:

1. מסקנה לגבי התרגיל, אם נרצה להעביר ערך מסוים למחסנית הפלט יש תחילה להעביר את כל הערכים הקטנים ממנו למחסנית העזר.

א. סדר הפעולות:

Push -> Push -> Push -> Push -> Push -> Pop -> Push -> Push -> Pop -> Push ->
-> Push -> Pop -> Pop -> Pop -> Pop -> Pop -> Push -> Pop -> Pop -> Pop

ב. סדר הפעולות:

Push -> Push -> Push -> Pop -> Push -> Push -> Push -> Pop -> Push -> Pop ->
-> Push -> Pop -> Pop -> Push -> Pop -> Push -> Pop -> Pop -> Pop -> Pop

ג. סדר הפעולות:

Push -> Push -> Push -> Push -> Push -> Pop -> Pop -> Pop -> Pop -> Pop ->
-> Push -> Pop -> Push -> Pop -> Push -> Pop -> Push -> Pop -> Push -> Pop

ד. סדר הפעולות:

Push -> Push -> Push -> Push -> Push -> Pop -> Pop -> Pop -> Pop -> Pop ->
Push -> Push -> Push -> Push -> Push -> Pop -> Pop -> Pop -> Pop -> Pop ->

ה. את המחסנית הבאה:

2	0	1	4	3	9	8	7	6	5
---	---	---	---	---	---	---	---	---	---

2. א. פסאודו קוד:

```

Josephus(n, k)
  Q = CreateQueue()
  for i = 1 to n do:
    Enqueue(i, Q)
  end for
  while IsEmpty(Q) = False do:
    for i = 1 to k - 1 do:
      Enqueue(Dequeue(Q), Q)
    end for
    Print Dequeue(Q)
  end while
  
```

ב. נחשב את תמורת יוספוס עבור $n = 9$ ו- $k = 2$:
 תחילה התור הוא: $Q = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$
 לאחר כל שלב הוצאה נקבל:

$\{1, 2, 3, 4, 5, 6, 7, 8, 9\} \Rightarrow (2) \{3, 4, 5, 6, 7, 8, 9, 1\} \Rightarrow (2, 4) \{5, 6, 7, 8, 9, 1, 3\} \Rightarrow$
 $\Rightarrow (2, 4, 6) \{7, 8, 9, 1, 3, 5\} \Rightarrow (2, 4, 6, 8) \{9, 1, 3, 5, 7\} \Rightarrow (2, 4, 6, 8, 1) \{3, 5, 7, 9\} \Rightarrow$
 $\Rightarrow (2, 4, 6, 8, 1, 5) \{7, 9, 3\} \Rightarrow (2, 4, 6, 8, 1, 5, 9) \{3, 7\} \Rightarrow (2, 4, 6, 8, 1, 5, 9, 7) \{3\} \Rightarrow$
 $\Rightarrow (2, 4, 6, 8, 1, 5, 9, 7, 3)$

ג. יש לעמוד במקום השלישי, זאת כיוון שהאיבר האחרון בסדרת יוספוס הוא 3.

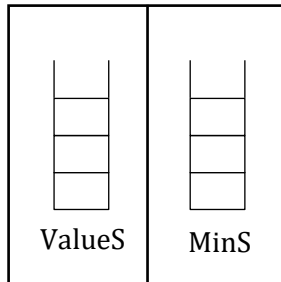
```
Alg(S1, S2)
  if (IsEmpty(S1) = True or IsEmpty(S2) = True) then:
    return -1
  sum1 = Pop(S1)
  sum2 = Pop(S2)
  while IsEmpty(S1) ≠ True or IsEmpty(S2) ≠ True do:
    if (sum1 < sum2) then:
      if (IsEmpty(S1) = True) then:
        return -1
      else then:
        sum1 = sum1 + Pop(S1)
    else if (sum1 > sum2) then:
      if (IsEmpty(S2) = True) then:
        return -1
      else then:
        sum2 = sum2 + Pop(S2)
    if (sum1 = sum2) then:
      return sum1
  end while
  return -1
```

ב. קטע הקוד נמצא במודל (לעיון בקטע הקוד יש להוסיף את ספריית "Stack.h" המצאת גם היא במודל).

```
Alg(Q1, Q2)
  while IsEmpty(Q1) = False and IsEmpty(Q2) = False do:
    if (Front(Q1) = Front(Q2)) then:
      return Front(Q1)
    if (Front(Q1) < Front(Q2)) then:
      Dequeue(Q1)
    else then:
      Dequeue(Q2)
  end while
  return -1
```

ב. קטע הקוד נמצא במודל (לעיון בקטע הקוד יש להוסיף את ספריית "Stack.h" המצאת גם היא במודל).

S:



5. נגדיר את מבנה הנתונים החדש MinStack בצורה הבאה:
MinStack ששמו S יחזיק שני מחסניות רגילות: ValueS ו-MinS:

פעולות בסיסיות כמו איתחול נבצע רגיל:
נאתחל גם את ValueS וגם את MinS.

כעת נגדיר את הפעולות Push, Pop, Min, PopMin:

Push(x, S) :

```

Push(x, S.ValueS)
if (IsEmpty(S.MinS) or x < Top(S.MinS)) then:
    Push(x, S.MinS)
else then:
    Push(Top(S.MinS), S.MinS)
  
```

Pop(S) :

```

Pop(S.MinS)
return Pop(S.ValueS)
  
```

Min(S) :

```

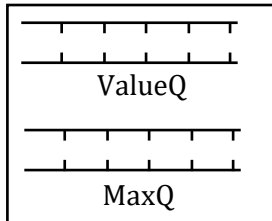
return Top(S.MinS)
  
```

PopMin(S)

```

tempS = CreateStack()
minimum = Top(S.MinS)
while Top(S.ValueS) ≠ minimum do:
    Push(Pop(S), tempS)
end while
Pop(S)
while IsEmpty(tempS) = False do:
    Push(Pop(tempS), S)
end while
  
```

Q: 6. נגדיר את מבנה הנתונים החדש MaxQueue בצורה הבאה:
 MaxQueue ששמו Q יחזיק שני תורים: ValueQ ו-MaxQ:



פעולות בסיסיות כמו איתחול נבצע רגיל:
 נאתחל גם את ValueQ וגם את MaxQ.

כעת נגדיר את הפעולות
 Enqueue, Dequeue, Max, DequeueMax:

```

Enqueue(x, Q)
  Enqueue(x, Q.ValueQ)
  tempQ = CreateQueue()
  while IsEmpty(Q.MaxQ) = False do:
    Enqueue(Dequeue(Q.MaxQ), tempQ)
  end while
  while IsEmpty(tempQ) = False do:
    y = Dequeue(tempQ)
    if (x > y) then:
      Enqueue(x, Q.MaxQ)
    else then:
      Enqueue(y, Q.MaxQ)
    end if
  end while
  Enqueue(x, Q.MaxQ)
  
```

```

Dequeue(Q)
  Dequeue(Q.MaxQ)
  return Dequeue(Q.ValueQ)
  
```

```
Max(Q)
    return Front(Q.MaxQ)

DequeueMax(Q)
    maximum = Front(Q.MaxQ)
    tempQ = CreateQueue()
    while Front(Q.ValueQ) ≠ maximum do:
        Enqueue(Dequeue(Q), tempQ)
    end while
    Dequeue(Q)
    while IsEmpty(Q.ValueQ) = False do:
        Enqueue(Dequeue(Q), tempQ)
    end while
    while IsEmpty(tempQ) = False do:
        Enqueue(Dequeue(tempQ), Q)
    end while
```

7. נממש את הפעולות הפשוטות של תור Q בעזרת שני מחסניות S1 ו-S2:

```
CreateQueue()
    Q.S1 = CreateStack()
    Q.S2 = CreateStack()
    Q.Size = 0
    return Q

Enqueue(x, Q)
    Push(x, Q.S1)
    Q.Size = Q.Size + 1

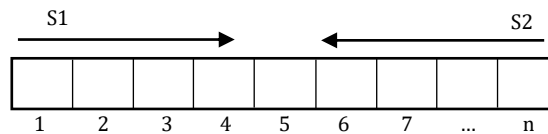
Dequeue(Q)
    if (IsEmpty(Q.S2) = True) then:
        then while IsEmpty(Q.S1) = False do:
            Push(Pop(Q.S1), Q.S2)
        end while
    Q.Size = Q.Size - 1
    return Pop(Q.S2)

Front(Q)
    if (IsEmpty(Q.S2) = True) then:
        then while IsEmpty(Q.S1) = False do:
            Push(Pop(Q.S1), Q.S2)
        end while
    return Top(Q.S2)

IsEmpty(Q)
    if (IsEmpty(Q.S1) = True and IsEmpty(Q.S2) = True) then:
        return True
    return False

Size(Q)
    return Q.Size
```


8. כן, ניתן לממש!
רעיון המימוש:



מערך A בגודל n ייצג לנו שני מחסניות, הראשונה $S1$ היא מתחילת המערך (אינדקס 1) ועד לסופו, והשניה היא מהאינדקס האחרון (n עד ולתחילת המערך).
כאשר האינדקסים המצביעים על ראשי המחסניות יהיו סמוכים זה לזה נדע שהמערך מלא ולכן גם המחסניות, במקרה זה הכנסת ערך נוסף למחסניות יגרום לגלישה.
המבנה יחזיק מערך A , גודלו n , אינדקס $S1Index$ לראש מחסנית $S1$ ואינדקס $S2Index$ לראש מחסנית $S2$.
נממש את הפעולות בצורה הבאה:

CreateStack(n)

$S.A = \text{CreateArray}(n)$

$S.n = n$

$S.S1Index = 0$

$S.S2Index = n + 1$

PushS1(x, S)

if ($S.S2Index \neq S.S1Index + 1$) then:

$S.S1Index = S.S1Index + 1$

$A[S.S1Index] = x$

else then:

Print " Overflow in Stack 1"

PushS2(x, S)

if ($S.S2Index \neq S.S1Index + 1$) then:

$S.S2Index = S.S1Index - 1$

$A[S.S2Index] = x$

else then:

Print " Overflow in Stack 2"

PopS1(S)

```
if (S.S1Index = 0) then:  
    Print "S1 is Empty"  
S.S1Index = S.S1Index - 1  
return A[S.S1Index + 1]
```

PopS2(S)

```
if (S.S2Index = S.n + 1) then:  
    Print "S2 is Empty"  
S.S2Index = S.S2Index + 1  
return A[S.S2Index - 1]
```

IsEmptyS1(S)

```
if (S.S1Index = 0) then:  
    return True  
else then:  
    return False
```

IsEmptyS2(S)

```
if (S.S2Index = S.n + 1) then:  
    return True  
else then:  
    return False
```